

A Large-Scale Benchmark and Risk Assessment of Traffic Analysis Attacks on Cloud LLM Services

Shahrooz Pouryousef*, Jesus Lopez[†], Saeefa Rubaiyat Nowmi[‡], Md Mahmuduzzaman Kamol[‡]

Moinul Hossain[§], Muoi Tran*, Mohammad Saidur Rahman[†]

*Chalmers University of Technology, Sweden [†]University of Texas at El Paso, USA

[‡]Old Dominion University, USA [§]George Mason University, USA

Abstract—Cloud-based Large language model (LLM) services create a network-level traffic side channel that can expose model, prompt, and task behavior despite encryption. From packet sizes, directions, timing, and burst structure alone, a passive local observer can infer the serving model, the user’s prompt category, and the task executed by a collaborative multi-agent system. Yet current evidence is fragmented across separate datasets and settings, limiting reproducibility and comparison. We present, to our knowledge, the first unified measurement study and public benchmark of encrypted LLM traffic across both user–LLM and multi-agent executions. The large-scale benchmark contains 60,000 user–LLM interactions across 10 models and 6 prompt categories, plus 2,838 multi-agent executions covering 10 task categories and two coordination topologies. Using only encrypted packet metadata, we assess the risk of traffic analysis attack by characterizing traffic signatures, identifying the features most associated with leakage, and testing robustness under prompt reformulation, decoding-temperature changes, larger candidate model sets, and partial traffic observation. Model fingerprinting achieves 97.7% balanced accuracy, prompt-category fingerprinting reaches 76.7% mean accuracy, and multi-agent task fingerprinting achieves up to 90.7% accuracy. Prompt reformulation weakens but does not remove model-specific leakage, and task fingerprints remain detectable even from a single agent’s traffic.

Index Terms—Cloud LLM Systems; Multi-Agent Systems; Traffic Analysis; Fingerprinting Attacks;

Availability. Code and data are available at <https://github.com/IQSeC-Lab/AI-Traffic-Analysis>.

I. INTRODUCTION

Modern large language models (LLMs) are deployed as networked applications. These applications include direct user–LLM interactions where users submit prompts to remote models and collaborative multi-agent workflows, in which multiple agents repeatedly invoke remote models, exchange intermediate results, and coordinate to complete tasks [1], [2]. We study both settings, focusing on the encrypted network traffic generated during remote LLM inference. Prior research on encrypted traffic analysis has shown that, although encryption prevents direct inspection of communication content, a passive on-path observer can exploit side-channel metadata, e.g., packet sizes, directions, timing, and burst structure, to infer application and user activities [3]–[5].

Networked LLM services inherit many security risks of distributed systems, including threats to confidentiality, integrity, and availability, but in this paper we focus specifically on privacy leakage to a passive on-path adversary. Such an adversary, such as a local eavesdropper, an ISP, or a network

middlebox, records packet metadata without modifying or decrypting the communication. It has been studied extensively in website fingerprinting and traffic analysis, where packet sequences and timing can reveal fine-grained user activity despite transport-layer encryption [3], [4], [6]–[9].

Motivation. Side-channel fingerprinting attacks have targeted LLM-based services too. Prior work has reconstructed portions of LLM responses and inferred their topics from token-length patterns exposed in encrypted traffic [10], [11] and exploited data-dependent inference timing to infer properties of user inputs [12]. For model identification, LLMmap actively probes applications and analyzes their outputs [13], whereas *LLMs Have Rhythm* passively fingerprints serving models using inter-token timing and encrypted network traffic [5]. Other studies infer prompt topics from packet sizes and timing in encrypted streaming responses [14], and use encrypted user–agent traffic to identify agent behaviors and agents and to profile user attributes from their inferred agent usage [15]. However, these efforts use separate datasets, collection setups, labeling schemes, and adversary assumptions, typically focusing on a narrow set of inference targets or interaction settings. This fragmentation makes it impossible to compare attacks and defenses under a common passive-observer model or to characterize privacy leakage across both direct user–LLM interactions and collaborative multi-agent workflows.

Shared benchmarks helped address this problem in related fields. Tor website-fingerprinting datasets allow researchers to compare attacks and defenses under common conditions, while GTT23 shows why realistic traces are important for measuring practical leakage [16]. Similarly, HELM, MMLU, RouterBench, and LLMRouterBench provide common tasks and evaluation settings for comparing LLMs and routing methods [17]–[20]. However, LLM traffic analysis lacks a similar foundation. To our knowledge, no public benchmark jointly covers direct user–LLM interactions and collaborative multi-agent executions while supporting model, prompt, and task inference under a common passive-observer model. This paper addresses that gap by constructing such a benchmark and assessing the risk of privacy leakage.

Challenges. Designing such a benchmark is challenging. It must capture real encrypted traffic from both direct user–LLM interactions and collaborative multi-agent workflows, span diverse models, prompts, and tasks, and provide reliable labels

at multiple levels of behavior, while restricting the observer to packet-level metadata available to an on-path adversary. For risk assessment, it must support controlled evaluation under changes to prompts, decoding settings, model sets, and traffic visibility to determine if observed leakage reflects persistent execution patterns or artifacts of a particular configuration.

Contributions. To address these challenges, we construct a unified benchmark of encrypted traffic from networked LLM services. It covers 60,000 direct user–LLM interactions across 10 models and 6 prompt categories, together with 2,838 successful multi-agent executions from MultiAgentBench [2] across 10 task categories and graph and star coordination topologies. All traces are collected and labeled through the same pipeline under a common passive-observer model, providing a consistent basis for studying leakage.

We use the benchmark to evaluate both established and less explored inference tasks. For established problems, the benchmark supports serving-model and prompt-category fingerprinting under a common evaluation setting. We also study multi-agent task fingerprinting, where an observer infers the collaborative task. These experiments show that the benchmark supports model-, prompt-, and task-level analysis and enables consistent evaluation across interaction settings.

Finally, we assess whether the observed leakage persists under changes in execution and observation. We vary prompt wording, decoding temperature, candidate model sets, and the amount of visible agent–LLM traffic. Model fingerprinting achieves 97.7% balanced accuracy, prompt-category fingerprinting reaches 76.7% mean accuracy, and multi-agent task fingerprinting achieves up to 90.7% accuracy. Prompt reformulation weakens but does not eliminate model-specific leakage, while collaborative tasks remain detectable even when the observer sees traffic from only one participating agent. These results show that the observed leakage persists beyond a single configuration. In summary, the key contributions are:

- **Unified encrypted-traffic benchmark.** We construct a benchmark of 60,000 user–LLM interactions and 2,838 multi-agent executions spanning diverse models, prompts, tasks, and coordination topologies.
- **Risk assessment across inference tasks.** We use the benchmark for serving-model and prompt-category fingerprinting and for the less-explored task of inferring multi-agent tasks. We also identify the features associated with each target.
- **Robustness under realistic variation.** We test prompt reformulation, temperature changes, larger model sets, and partial traffic visibility, showing when attacks weaken and when leakage persists.

II. RELATED WORK

Encrypted-traffic fingerprinting. Encrypted connections expose metadata, e.g., packet sizes, directions, timing, and burst structure. Website fingerprinting (WF) showed that these signals can identify user activity even over Tor [6], [8], [21]. Early attacks relied on hand-crafted features, whereas deep-learning methods learned representations directly from packet sequences and timing [3], [4], [9], [22]. Later studies examined

actual traffic, changing network conditions, and adaptive defenses, showing that lab performance may not transfer directly to realistic settings [23]–[25]. This literature motivates our use of encrypted packet metadata, but our objective is to infer properties of LLM computation rather than visited websites.

Traffic analysis of LLM services. Autoregressive generation and response streaming create observable patterns that reveal conversation content and serving-model properties. Weiss [11] reconstructs generated text and infers conversation topics from encrypted response sizes. Whisper Leak infers target topics from packet sizes and timing, while NetEcho reconstructs prompts and responses using offline probing and trace–text matching [14], [26]. Alhazbi [27] shows that inter-token timing produces model-specific signatures, LLMmap identifies serving models using an active adversary with plaintext responses [13], and Carlini and Nasr expose remotely observable timing leakage caused by data-dependent inference optimizations [12]. These works demonstrate leakage of topics, content, model identity, and serving behavior, but typically study a single target. Our benchmark instead supports serving-model and multiclass prompt-category fingerprinting under a common passive-observer model and evaluates robustness to prompt reformulation, decoding temperature, larger candidate sets, and partial traffic visibility.

Traffic analysis of LLM agents. Agentic systems generate rich traffic through planning, tool use, retrieval, and repeated model calls. AgentPrint analyzes encrypted user–agent traffic to identify agent activities and applications and infer user attributes from longitudinal usage patterns [15]. PersonaFingerprint shows that encrypted traffic produced by LLM-driven browsing can expose high-level user personas [28]. These studies examine individual agents or their external browsing behavior. In contrast, we study encrypted traffic jointly generated by collaborative multi-agent executions and assess if a passive observer can infer the task being performed. We evaluate graph and star coordination topologies and partial visibility, where only a subset of agent–LLM flows is observed.

III. PROPOSED THREAT MODEL

A. System and Observation Model

Figure 1 summarizes the two considered settings. In the direct setting, a user communicates with a remote LLM service over TLS. A passive on-path adversary records the encrypted traffic exchanged during each interaction; it cannot decrypt or modify the traffic, and neither the endpoint is compromised.

In the collaborative setting, multiple LLM-based agents¹ solve a shared task using MARBLE [2] under its star and graph-mesh coordination protocols. The topology of the coordination can be star, i.e., a central planner assigns subtasks and combines agent outputs, or mesh, i.e., agents communicate directly. Agents may communicate with the same or different remote LLM services. In both settings, the adversary observes

¹An LLM agent is a software system that combines one or more LLMs with tools, memory, and workflow logic to autonomously plan and execute tasks. Multi-agent systems comprise multiple collaborating LLM agents.

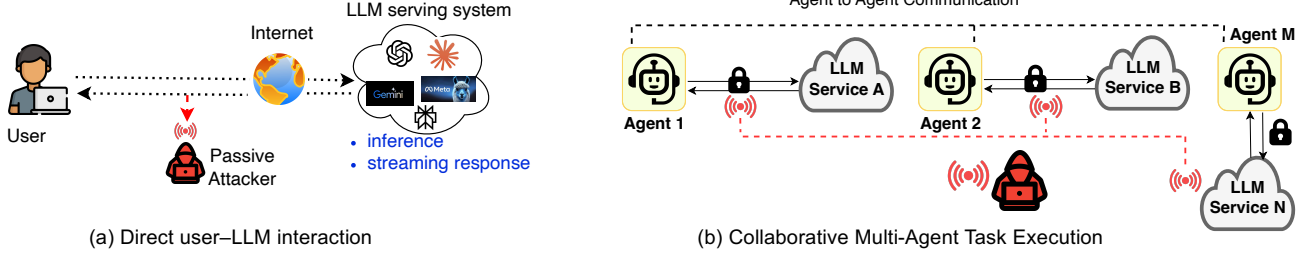


Fig. 1: Figure 1(a) illustrates the direct user-LLM setting. Figure 1(b) illustrates the collaborative multi-agent setting, where the adversary observes encrypted agent-LLM traffic but not agent-to-agent communication.

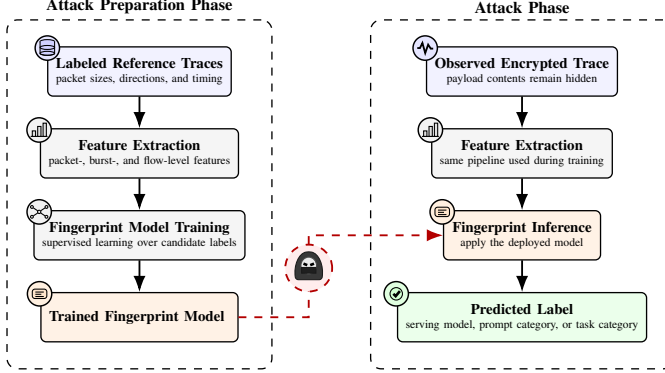


Fig. 2: **Workflow of the traffic-analysis attack.** The adversary trains a fingerprint model via labeled encrypted traffic traces and applies it to observed encrypted traffic to infer the serving model, prompt category, or collaborative task.

only packet timestamps, sizes, and directions. Packet payloads, prompts, responses, IP addresses, and endpoint identities remain hidden. In the collaborative setting, agent-to-agent communication is also unobservable. An observed traffic trace is represented as $F = \{(t_i, s_i, d_i)\}_{i=1}^n$, where t_i , s_i , and d_i denote the relative timestamp, size, and direction of packet i , respectively. We assume the adversary can correctly segment each interaction, direct or collaborative.

B. Adversary Knowledge, Objectives, and Scope

We consider a supervised closed-world setting. The adversary has access to labeled reference traces for the candidate labels, trains fingerprinting models offline, and predicts the label of an observed trace. For direct user-LLM interactions, the candidate labels correspond to serving models or prompt categories. The adversary seeks to infer $f_{\text{model}} : F \rightarrow \mathcal{M}$, $f_{\text{category}} : F \rightarrow \mathcal{P}$, where $\mathcal{M}$ and $\mathcal{P}$ denote the candidate model and prompt-category sets, respectively. For collaborative executions, the candidate labels correspond to collaborative task categories. The adversary seeks to infer $f_{\text{task}} : F_{\text{multi}} \rightarrow \mathcal{T}$, where F_{multi} denotes the aggregated encrypted traffic observed from the monitored agents during a single execution, and $\mathcal{T}$ is the set of candidate task categories. Coordination topology is not considered an inference target.

C. Traffic Fingerprinting Workflow

Figure 2 illustrates the considered traffic fingerprinting workflow. Under the closed-world assumption, the adversary first collects labeled reference traces for the candidate classes and trains a supervised classifier offline. Each traffic trace is segmented into an individual user-LLM interaction or collaborative multi-agent execution, from which packet-level and flow-level features are extracted to construct a feature vector. During deployment, the adversary observes an encrypted traffic trace from an unknown interaction, applies the same segmentation and feature extraction procedure, and provides the resulting feature vector to the trained classifier. Depending on the inference objective, the classifier predicts the serving model, the prompt category, or the collaborative task category. This workflow is common to all experiments, only the prediction target changes across the different inference tasks.

IV. PROPOSED EXPERIMENTAL METHODOLOGY

This section describes the generation of the user-LLM and multi-agent datasets, the traffic collection and preprocessing pipeline, and the extracted traffic features.

A. Dataset Generation

User-LLM Dataset. The dataset contains 60 prompts evenly distributed across 6 semantic categories: summarization, code generation, mathematical reasoning, logical reasoning, technical explanation, and adversarial requests. Examples of prompts include “Implement binary search in Python” for code generation, and “If you have a drawer full of unmatched socks, how many do you have to pull out to get a matching pair?” for logical reasoning. Each prompt is independently submitted to every candidate LLM over 100 fresh sessions. The framework automatically records the model identity, prompt ID, prompt category, and trial ID as ground-truth labels.

Multi-Agent Dataset. We construct the multi-agent dataset using MARBLE [2], where teams of LLM-based agents collaborate on shared tasks. It includes 10 categories with 15 tasks per category. Four categories are native to MARBLE: bargaining, research, coding, and database. We additionally incorporate bug fixing from TeamBench [29], software-engineering issue resolution from SWE-bench Lite [30], long-form report writing from DeepResearch Bench [31], clinical reasoning from MedQA [32], contract analysis from LegalBench [33], and argumentation based on competitive debate motions [34].

Each task is executed under two coordination topologies. In the fully connected graph, agents communicate directly with all peers; in the star topology, a central planner coordinates all communication. Teams contain 2–5 role-specialized agents. Execution ends when the planner declares completion or the configured round limit is reached. All agents use the same locally hosted Llama 3.2 3B model via Ollama, isolating task and topology effects from model variation.

Each task is independently repeated up to 15 times to capture variation from stochastic decoding. The database tasks additionally interact with a containerized PostgreSQL system containing realistic performance anomalies. Overall, we obtain 4,070 successful and 42 failed executions, corresponding to a 98.9% success rate. Of the successful executions, 3,970 contain valid encrypted-traffic captures and are retained.

Task categories lead to different communication structures, varying in coordination rounds, LLM request/response size, and the amount of tool or environment activity between calls. These workflows create distinct packet count, size, burst, and timing patterns without requiring access to encrypted content.

B. Traffic Collection and Preprocessing

Traffic is captured with `tcpdump` at the earliest observable point on the network path, i.e., the client-facing interface for single-agent interactions and the proxy-facing loopback interface for multi-agent executions. Each capture is synchronized with its corresponding execution via recorded start and end timestamps, allowing a continuously running capture to be sliced into per-execution traces after the fact, rather than requiring the capture tool to be restarted for every interaction.

Each trace is anonymized and preprocessed before feature extraction: packets are restricted to the execution’s time window, timestamps are normalized relative to the first packet of the trace, and direction is labeled outgoing/incoming from the fixed client/server role of the connection endpoints. Byte-identical duplicate packets, arising when overlapping capture processes observe the same traffic, are removed by content hashing; genuine TCP retransmissions are rare under our capture conditions and are left unmodified where they occur. Executions whose capture failed to record any packets due to transient tooling faults were discarded and re-executed.

C. Experimental Environment

The experimental pipeline, including local LLM inference, packet capture, PCAP processing, and feature extraction, was executed on a high-performance computing server equipped with two Intel Xeon Gold 6430 processors (64 physical cores, 128 logical cores) and 512 GB of RAM. The server contained two NVIDIA H100 NVL GPUs, each with 95.8 GB of memory. GPU-accelerated workloads were executed using CUDA 12.8 and NVIDIA driver version 595.71.05.

V. RISK ASSESSMENT AND BENCHMARK EVALUATION

In this section, we empirically evaluate the encrypted traffic fingerprinting attacks introduced in Section III. First, we characterize the encrypted traffic generated by different models, prompt categories, and collaborative multi-agent tasks

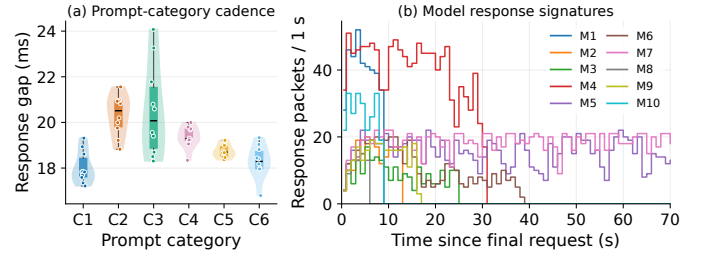


Fig. 3: Encrypted user-LLM traffic properties. (a) Response-packet inter-arrival timing across prompt categories for a fixed LLM. Each point represents one prompt; boxes show medians and interquartile ranges. (b) Temporal response-packet signatures of 10 LLMs for a fixed prompt. Each line is the execution closest to the model’s median behavior over 100 trials. Payload-bearing LLM-to-user packets are aggregated into 1 s intervals.

to understand whether they exhibit distinguishable communication patterns. Second, we assess the risk of encrypted traffic fingerprinting for model, prompt-category, and multi-agent task identification. Third, we investigate which observable traffic characteristics are responsible for the information leakage. Finally, we evaluate the practicality of the proposed fingerprinting attack.

Unless stated otherwise, all experiments use a Random Forest (RF) classifier. We report balanced accuracy (BA) as the primary evaluation metric together with the macro-averaged F1 score (Macro-F1). Balanced accuracy is computed as the average recall across all C classes,

$$BA = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i}, \quad (1)$$

where TP_i and FN_i denote the true positives and false negatives of class i , respectively. The Macro-F1 is defined as

$$\text{Macro-F1} = \frac{1}{C} \sum_{i=1}^C \frac{2P_i R_i}{P_i + R_i}, \quad (2)$$

where P_i and R_i denote class i ’s precision and recall.

A. User-LLM Fingerprinting

Key Insight: One Traffic Stream, Two Leakage Sources

The serving model shapes how the response is streamed, while the prompt category shapes how much and how long the model generates. One passive capture therefore supports two inference attacks simultaneously, so we assess each risk in isolation by fixing the prompt when fingerprinting models and fixing the model when fingerprinting prompt categories.

We evaluate encrypted traffic generated by direct interactions with 10 LLMs downloaded from Hugging Face: Llama-3.2-3B-SRL, Qwen2.5-7B-Instruct, Qwen3-14B-Base, MegaScience-Qwen3-14B, Phi-4-Mini-Reasoning, AceReason-Nemotron-1.1-7B, PatronusAI-Glider, LLMTwin-Llama-3.1-8B, Typhoon2-8B-Instruct, and XtraGPT-3B. We evaluate 10 more models to assess scalability. Unless otherwise noted, all experiments use traffic from the original 10 models.

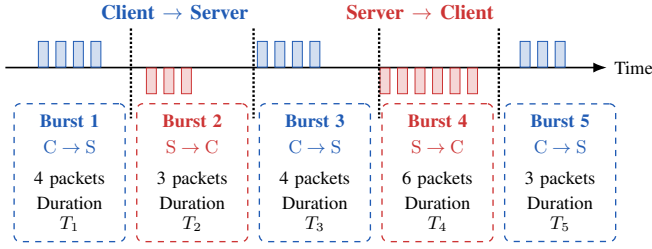


Fig. 4: **Burst-level traffic representation.** Consecutive packets transmitted in the same direction form a burst.

1) *Encrypted Traffic Leaks Distinguishable Signals:* We first examine whether encrypted traffic generated by different LLMs exhibits distinguishable statistical patterns. For each PCAP trace, our feature-extraction pipeline computes 176 content-independent traffic statistics from packet direction, encrypted payload length, arrival time, rate, bursts, idle periods, and packet ordering. We exclude IP addresses, endpoint identifiers, payload contents, and other application metadata. After removing 8 constant features, each interaction is represented by a 168-dimensional feature vector. A directional burst is a maximal sequence of consecutive payload-bearing packets traveling in the same direction,

$$B = (p_i, \dots, p_j),$$

whose duration is

$$D(B) = t_j - t_i.$$

Figure 3(a) shows that prompt category affects encrypted response timing even for a fixed LLM. Categories C2 and C3 generally exhibit longer inter-packet gaps than C1 and C6, while C3 shows greater variation, consistent with math and algorithmic prompts requiring different generation effort. However, the figure cannot separate token-generation time from serving and packetization effects. Figure 3(b) shows the complementary model-dependent effect for a fixed prompt: models differ in packet rates, response durations, and streaming patterns. These distinct timing and streaming signatures are consistent with prior work [5], [15], [35].

2) *LLMs Can Be Reliably Fingerprinted:* Since different models exhibit distinguishable traffic characteristics, we next examine which features contribute most to model fingerprinting. We group the 168 extracted features into six families by representation granularity: flow-level *volume/direction* and *rate*, packet-level *packet size* and *timing*, segment-level *burst*, and sequence-level *ordering*. Flow-level features summarize aggregate volume, directionality, and transmission rates, while packet-level features capture individual packet sizes and temporal relationships. Burst features characterize same-direction transmission segments, where a burst is a maximal sequence of consecutive payload-bearing packets transmitted in one direction, as illustrated in Fig. 4. These features summarize burst count, size, duration, and inter-burst spacing. Ordering features instead capture the global packet arrangement, including packet positions, early- and late-stage traffic concentration, response concentration, and sequence-order statistics.

TABLE I: Feature-family analysis for LLM-model fingerprinting using one repetition per model-prompt pair. Values are percentages (mean $\pm$ std dev across five prompt-grouped folds).

Feature family	No. features	Family only (%)	Without family (%)
All features	168	95.67 $\pm$ 0.70	–
Burst	75	94.17 $\pm$ 1.56	86.33 $\pm$ 2.67
Timing	28	66.50 $\pm$ 1.09	95.67 $\pm$ 1.49
Packet size	21	87.83 $\pm$ 1.62	95.83 $\pm$ 1.32
Ordering	22	48.00 $\pm$ 5.67	95.83 $\pm$ 0.83
Volume/direction	10	30.17 $\pm$ 4.87	96.33 $\pm$ 1.12
Rate	12	47.83 $\pm$ 4.51	97.33 $\pm$ 1.09

Key Insight: Burst Features Dominate

Burst features alone identify the model nearly as well as all 168 features (94.2% vs. 95.7%), showing that response packetization and streaming (not traffic volume) carry the signal. This lowers the cost of the attack, since coarse burst statistics are sufficient. It also implies that defenses hiding only a single feature such as packet size or timing leave the dominant side-channel open.

For each family, we perform two complementary experiments. In the *family-only* setting, the classifier is trained using only that family to measure its predictive sufficiency. In the *without-family* setting, the family is removed and the classifier is retrained using all remaining features to measure its unique contribution. We train a new RF for every feature configuration and prompt-grouped fold using the same protocol as the main evaluation, with preprocessing fitted only on the training partition. Table I reports the mean and standard deviation across the five test folds.

The feature families contain different numbers of features. Consequently, family-only accuracy reflects the predictive sufficiency of the complete family rather than a cardinality-controlled comparison. We therefore use the *without-family* results to assess each family’s unique contribution.

Burst alone could be enough. Using all 168 features, the classifier identifies the deployed model with 95.67% accuracy, well above the 10% random-guessing baseline. Burst features achieve 94.17% in isolation, and removing them reduces accuracy by 9.33%. A directional burst combines the volume and duration of consecutive packets traveling in one direction with their separation from adjacent transmissions. It therefore captures both the packetization and temporal structure of a streamed response. Packet-size and timing features are also informative in isolation, achieving 87.83% and 66.50%, respectively, but contribute little beyond burst features because they capture overlapping information. In contrast, aggregate volume and direction are much less effective because they discard the packetization and temporal structure of the response.

Why do these features work? Two mechanisms explain why these features differ across models. First, autoregressive decoding time depends on the deployed model’s architecture, parameter count, attention configuration, and execution characteristics. Prior work shows that these differences produce model-dependent inter-token timing patterns observable after responses are transmitted over a network [5]. Our evaluated models span approximately 3B–15B parameters and several Llama, Phi, and Qwen architectures, providing substantial

variation in the computation performed at each decoding step.

Key Insight: Fingerprints Reflect the Deployed LLM System

The fingerprint identifies the deployed serving system rather than architecture alone. Two post-trained models sharing the Qwen3-14B backbone remain separable, with 100% and 96.7% recall, while the most common confusions occur across different architectures. Thus, withholding model details does not prevent passive identification of serving variants.

Second, training, post-training, alignment, and tokenization affect what each model generates for the same prompt. LLMmap demonstrates that identical queries elicit model-dependent response lengths, structures, wording, and refusal behavior [13]. These textual differences become network-visible in our collection pipeline. The server streams each decoded text chunk to the client as a separate Server-Sent Events (SSE) message over a persistent HTTP connection. Consequently, differences in tokenization, word length, formatting, and response structure alter the sizes and sequence of transmitted chunks, while differences in decoding speed alter their emission intervals.

Architecture alone, however, does not explain the result. Qwen3-14B-Base and MegaScience-14B share the same Qwen3-14B backbone, but the latter is further trained for scientific reasoning; the classifier obtains 100% and 96.7% recall for these models, respectively. This indicates that post-training-induced response behavior can change the traffic fingerprint without changing the base architecture. Conversely, most classification errors occur between Phi-4-Mini and LlamaTwin-8B, showing that models with different architectures can still produce overlapping traffic behavior. We therefore interpret the measured signature as a fingerprint of the deployed LLM serving system rather than the model architecture alone. The fingerprint reflects the combined effects of the model architecture, post-training, tokenization, and generation behavior.

3) *Prompt-category fingerprinting*: We next investigate whether encrypted traffic reveals the semantic category of a prompt when the deployed LLM is fixed. We train a separate six-class Random Forest classifier for each of the ten LLMs, preventing model-specific traffic differences from directly determining the predicted category. We use all available traces and five common prompt-grouped folds. In each fold, the classifier for a given model is trained on all repetitions of 48 prompts and tested on all repetitions of 12 completely unseen prompts. All repetitions of a prompt remain in the same fold. We first average accuracy across the five folds for each model and then report the mean and standard deviation across the ten models. Random guessing obtains 16.67%.

Response-side prompt-category fingerprinting. Prompt-category classification can be biased by encrypted request traffic because categories differ in prompt length. Although each prompt fits within one request packet, median encrypted sizes vary widely across C1–C6: 1,388, 131, 77.5, 146.5, 283, and 95.5 bytes. Accordingly, Table II shows that request traffic alone achieves 51.68% accuracy, revealing prompt length as a

TABLE II: Prompt-category fingerprinting using different traffic views and response-only feature families. “Only” uses only the indicated response-feature family, whereas “Without” removes it from the complete 22-feature response representation. Values are mean accuracy across the ten LLM-specific classifiers.

Features	No.	Only (%)	Without (%)	Drop (pp)
<i>Traffic-view sanity check</i>				
All traffic	168	65.10 $\pm$ 7.93	–	–
Request only	16	51.68 $\pm$ 0.81	–	–
Response only	22	67.37 $\pm$ 11.24	–	–
<i>Response-only feature families</i>				
Volume	2	37.12 $\pm$ 7.08	67.32 $\pm$ 11.37	0.05
Packet size	7	49.11 $\pm$ 9.16	59.03 $\pm$ 8.82	8.34
Timing	8	55.40 $\pm$ 9.44	60.11 $\pm$ 10.19	7.26
Rate	2	42.23 $\pm$ 7.99	67.37 $\pm$ 11.29	0.01
Temporal ordering	3	36.63 $\pm$ 5.78	65.63 $\pm$ 12.02	1.74
Random guessing	–	16.67	–	–

TABLE III: Per-category performance using response-only traffic and all repetitions. Metrics are computed from pooled out-of-fold predictions. The final column reports the most frequent wrong prediction as a percentage of the true category’s traces.

Category	Precision	Recall	F1	Main confusion
C1	75.65%	79.91%	77.72%	C4 (7.92%)
C2	58.31%	52.82%	55.43%	C3 (16.21%)
C3	66.74%	67.10%	66.92%	C2 (15.07%)
C4	69.07%	73.15%	71.05%	C6 (9.14%)
C5	66.08%	59.57%	62.66%	C6 (12.11%)
C6	66.37%	71.15%	68.68%	C4 (8.72%)

strong shortcut. Because prompt length can be altered without changing task semantics, we focus on the more challenging response-only setting using 22 LLM-to-user features.

Key Insight: Categories Leak Through Generation Behavior

With request packets excluded and all test prompts held out, response packetization and timing alone identify the prompt category with 67.4% accuracy, compared with a 16.7% baseline. This reveals the type of task a user is performing even when prompt length changes, because the leakage originates from the generated response rather than the request.

Using only response traffic, Table II achieves 67.37% accuracy, exceeding both the request-only (51.68%) and all-traffic (65.10%) settings. Timing is the strongest feature family in isolation (55.40%), while packet size contributes the most unique information, reducing accuracy by 8.34 percentage points when removed, followed by timing (7.26 points). Removing volume or rate causes almost no degradation, indicating that they are largely redundant with the more detailed packet-size and timing statistics. Because request-side features are excluded and complete prompts are held out, the observed 67.37% accuracy cannot be explained by prompt length. Instead, different prompt categories induce different response structures and decoding behaviors that, in our SSE-based serving pipeline, translate into distinct encrypted chunk sizes and transmission intervals, consistent with prior work [13], [35]. Overall, prompt-category leakage is driven primarily by response packetization and generation timing rather than aggregate response volume.

Table III reports the per-category precision, recall, F1 score, and most frequent misclassification using response-only

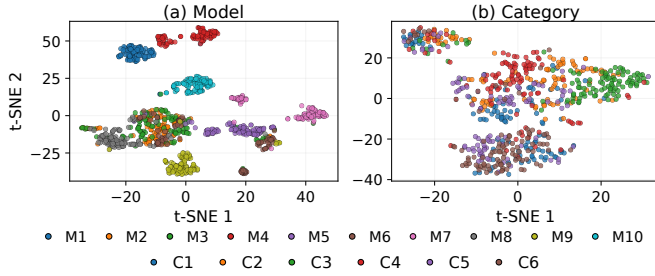


Fig. 5: t-SNE visualization of encrypted user-LLM traffic. Panel (a) compares ten LLMs executing the same prompt using all 168 features. Panel (b) compares six prompt categories for a fixed LLM using all 22 response-side features. Each point represents one traffic trace; colors denote models or prompt categories.

traffic. C2 and C3 are most frequently confused, because code generation and mathematical reasoning often produce similarly structured multi-step outputs. To investigate this, we compare category distributions using the 22 response-only features. Each model-prompt pair is represented by the median over its repetitions, robustly normalized within each model, and compared using the Wasserstein distance. Among all 15 category pairs, C2 and C3 have the smallest average distance (0.786), confirming that their responses are the most similar. This similarity is driven by timing: they have the closest mean response inter-arrival time and the second-closest median inter-arrival time, packet rate, and byte rate, whereas response length, byte volume, and duration differs substantially.

4) *Feature-space Visualization*: Figure 5 provides a qualitative visualization of the feature spaces for model and prompt-category fingerprinting. The model embedding exhibits clearer local separation than the prompt-category embedding, consistent with the corresponding classification accuracies (95.67% and 67.37%). Because t-SNE preserves local neighborhoods but distorts global distances, it is intended only as a qualitative illustration rather than an objective measure of performance.

5) *Information Leakage Is Concentrated in a Few Feature Families*: The preceding experiments assess LLM fingerprinting using Random Forest classification. As accuracy depends on the chosen classifier and does not directly quantify the information revealed about LLM identity, we complement it with an information-theoretic analysis. Following [36], we measure leakage as the mutual information between the observed traffic features (X) and LLM identity (M).

We retain 59 response-side features and estimate their mutual information with model identity using WeFDE [36]. Under a uniform prior over ten models, the maximum leakage is ($H(M) = \log_2 10 = 3.32$) bits. Within each of five prompt-grouped folds, we estimate per-feature leakage using adaptive kernel density estimation and 5,000 Monte Carlo samples, remove features with normalized mutual information above 0.9, and combine the remaining class-conditional marginal likelihoods under a conditional-independence approximation. All estimation, feature selection, and density fitting use only the training prompts.

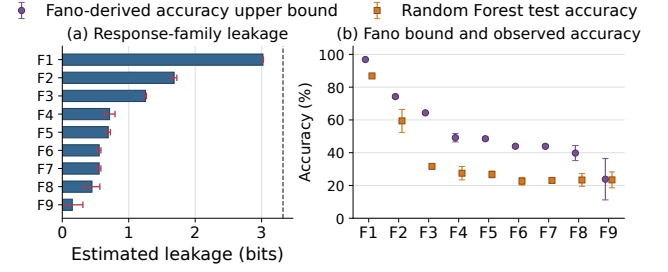


Fig. 6: Response-side model leakage and fingerprinting performance. Families are ordered by estimated leakage, with F1-F9 defined in the legend. (a) Response-family leakage $I(X; M)$; the dashed line marks the maximum possible leakage $H(M) = 3.32$ bits. (b) Fano-derived accuracy upper bound and held-out Random Forest test accuracy. Error bars denote 95% confidence intervals across five prompt-grouped folds.

Key Insight: Information-Theoretic Analysis Confirms Burst Dominance

Classifier-free analysis confirms the ranking: burst-byte features leak 3.02 of the maximum 3.32 bits about model identity and alone support 86.9% accuracy, while total response volume yields only 23.1%. Because the leakage is a property of the encrypted traffic rather than of any particular classifier, the risk persists and can only be reduced by reshaping the traffic profile itself.

We relate leakage to classification using Fano’s inequality [37],

$$H(M | X) \leq h_2(P_e) + P_e \log_2(9). \quad (3)$$

Solving for (P_e) yields a lower bound on classification error and, equivalently, an upper bound $(1 - P_e^{\text{Fano}})$ on accuracy. Because this bound is generally loose and not Bayes-optimal, we compare it with a Random Forest trained on the same selected features and evaluated on held-out prompts.

Figure 6 shows that response burst-byte features leak (3.02) bits and achieve 86.85% test accuracy, compared with 23.08% using total response volume alone. While total volume varies with the prompt, burst statistics capture how encrypted responses are segmented into application exchanges. Notably, the minimum response-burst size has zero within-model interquartile range and assumes eight distinct values across the ten models, consistent with model-dependent streaming, termination, or metadata records in the serving stack. Because payloads are encrypted, we interpret this as a deployed-system fingerprint rather than evidence that model architecture alone determines packetization.

Response inter-arrival features rank second, leaking 1.68 bits and achieving 59.4% accuracy. Their model-level medians range from approximately 19 to 52 ms, reflecting differences in generation and stream-delivery intervals, although overlapping distributions limit classification. Unlike burst-byte features, which exhibit near-zero within-model variance and behave almost categorically, inter-arrival timing varies continuously with decoding speed and network delivery. Rate features leak 1.25 bits but achieve only 31.65% accuracy, while packet-size features reach 23.35% because they discard burst structure

TABLE IV: Top-ranked task-fingerprinting features. The 75th percentile is denoted by Q_{75} ; outgoing and incoming indicate agent-to-LLM and LLM-to-agent traffic.

Rank	Graph	Star
1	Outgoing burst bytes, Q_{75}	Outgoing burst bytes, Q_{75}
2	Outgoing packet size, std.	Burst bytes, median
3	First-30 outgoing bytes	Outgoing packet size, mean
4	Incoming burst bytes, median	Incoming burst bytes, median
5	Burst bytes, Q_{75}	Outgoing packet size, Q_{75}
6	Outgoing burst bytes, std.	Outgoing packet size, std.
7	Outgoing packet size, mean	Packet size, Q_{75}
8	Burst bytes, median	First-30 outgoing bytes
9	Outgoing packet size, Q_{75}	Burst bytes, Q_{75}
10	Outgoing burst bytes, maximum	Activity-episode bytes, minimum

and overlap under common transport segmentation. The gap between the Fano bound and observed accuracy reflects bound looseness and finite-sample leakage estimation, not distance from Bayes-optimal performance.

B. Encrypted Traffic Reveals Multi-Agent Tasks

Key Insight: Workflow Alone Leaks Task Identity

Even after removing all packet-size, byte-volume, and burst-byte features, workflow features alone identify the collaborative task at 54.0% (graph) and 58.7% (star) balanced accuracy against the 10% chance level, rising to 85.3% and 86.0% with all 247 features. For deployments of collaborative multi-agent systems, coordination structure is itself a sensitive attribute, so packet-size padding alone would not conceal what agents are doing.

We next evaluate whether encrypted agent-LLM traffic is sufficient to identify the task performed by a collaborative multi-agent system. We first characterize the traffic generated by different tasks and then measure fingerprinting performance under different communication topologies. Finally, we evaluate the attack when the observer captures encrypted traffic from only a subset of the agent-LLM communication channels, rather than all agents participating in the execution.

Tasks Produce Distinct Workflow Signatures. We next rank features using Random Forest mean decrease in impurity, computed only on the training partitions of five task-grouped folds. Table IV and Fig. 7(a) show that 8 of the 10 highest-ranked features are shared by both graph and star topologies. Burst volume, packet size, and early outgoing traffic dominate both, indicating that task identity is reflected consistently in response packetization and execution structure.

To isolate workflow structure from response size, we exclude packet-size, byte-volume, and burst-byte features, retaining 134 timing, burst, idle, directional, and progression features. The top 100 workflow features achieve 54.0% balanced accuracy under graph topology and 58.7% under star, well above the 10% chance level. Figure 7(b) shows partial category overlap, reflecting shared plan-execute-review workflows across tasks. Using all 247 features raises balanced accuracy to 85.3% and 86.0%, with macro-F1 scores of 84.6% and 85.2%, respectively. Thus, workflow structure alone reveals task identity, while response volume and packetization provide substantial complementary leakage.

Task fingerprints persist under topology changes and partial visibility. We evaluate whether task fingerprints transfer across

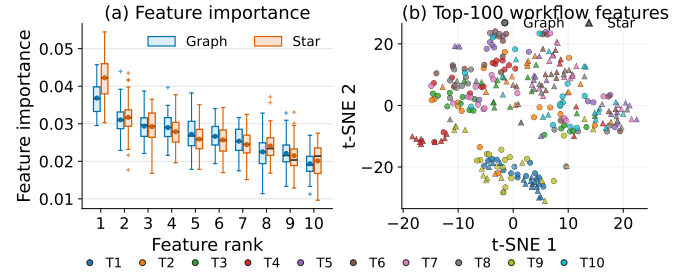


Fig. 7: Multi-agent task-fingerprinting features. (a) Importance of the 10 highest-ranked features under graph and star topologies. (b) Joint t-SNE using the 100 highest-ranked workflow-only features. Colors denote task categories; marker shapes denote topology.

agent-coordination topologies and remain effective when only a subset of agent traffic is observable.

Key Insight: Partial Visibility Still Leaks Tasks

From a single agent's traffic, an adversary infers the collaborative task with 66.5% balanced accuracy under graph topology and 78% under star topology, increasing to 84% and 86% with visibility into up to four agents. Cross-topology accuracy drops from 83% to 52% but remains well above random guessing. Thus, partial agent-LLM traffic visibility poses substantial privacy risk, while topology changes reduce but do not eliminate the leakage.

For topology transfer, we use feature sets of 3, 6, 10, 50, 100, and all 247 features. For each size below 247, we sample 20 label-independent feature subsets and train a new Random Forest for every subset and task-grouped fold. Within-topology evaluation trains and tests on the same topology, while cross-topology evaluation trains on graph and tests on star, or vice versa. For partial visibility, we construct nested traces containing traffic from up to k agents. We sample ten agent orderings per execution and retrain the classifier for every topology, visibility level, ordering, and fold. Agent identities are used only to construct the restricted traces and are never provided to the classifier.

Figure 8(a) shows that additional features improve task fingerprinting in both settings. With all features, within-topology accuracy reaches 83.0%, whereas cross-topology accuracy reaches only 52.0%. This transfer gap indicates that the fingerprint depends partly on the agents' coordination structure. A classifier trained on one topology therefore does not observe exactly the same traffic signature under another. The gap likely arises because topology shapes the observable traffic: the star topology routes coordination through a central hub, whereas the graph topology distributes it across peer-to-peer links, causing the classifier to learn topology-specific as well as task-dependent patterns. Figure 8(b) shows that restricted traffic visibility does not eliminate task leakage. Observing one agent yields 66.5% accuracy under graph and 78.0% under star. Accuracy increases to 84.3% and 85.9%, respectively, when traffic from up to four agents is visible. Thus, an individual agent's interactions already reveal a substantial portion of the task signature, while additional agents expose complementary workflow information and strengthen the attack.

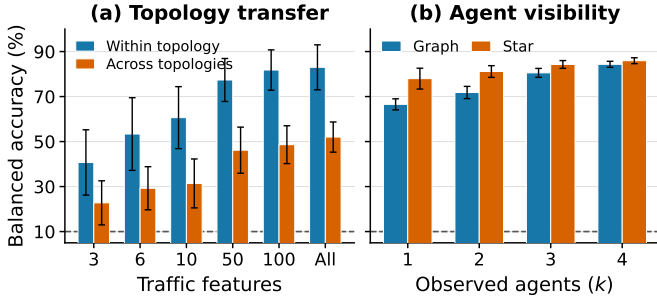


Fig. 8: Multi-agent task fingerprinting under topology changes and partial traffic visibility. (a) Within-topology and cross-topology accuracy as the number of traffic features increases. (b) Accuracy when the attacker observes traffic from up to k agents. Error bars summarize variation across feature subsets and agent selections, respectively. The dashed line denotes the 10% random-guessing baseline.

C. Fingerprinting Persists Under Realistic Conditions

Key Insight: Lightweight Countermeasures Reduce but Do Not Eliminate Fingerprinting

Prompt reformulation reduces model-fingerprinting accuracy by 24% and temperature randomization by up to 18%, but performance remains well above the 10% random baseline. Expanding the candidate set from 5 to 20 LLMs lowers accuracy only from 98.5% to 92%. Thus, from a risk perspective, lightweight defenses reduce but do not eliminate leakage risk, and model fingerprints remain highly discriminative even at larger scale.

1) *Lightweight Countermeasures*: We evaluate two lightweight countermeasures that require no network-protocol changes. Prompt reformulation rewrites or wraps the user’s request while preserving its semantics; we use the LLMmap strategy, which inserts execution-trigger instructions before and after the original prompt. Decoding variation instead changes the temperature across requests rather than using a fixed setting. As shown in Figure 9(a), prompt reformulation reduces model-fingerprinting accuracy from 93.2% to 69.5%, while temperatures of 0.3, 0.5, and 1.5 yield 75.0%, 88.8%, and 80.0%, respectively. Both countermeasures weaken fingerprints learned under a fixed configuration, but accuracy remains well above the 10% random baseline.

2) *Scalability*: We next evaluate model fingerprinting as the candidate set grows from 5 to 20 LLMs. Figure 9(b) compares our 168-feature representation with a 35-feature baseline using a matched experimental protocol. The proposed representation maintains high accuracy as the number of candidate models increases, decreasing from 98.5% with five models to 92.2% with twenty models. In contrast, the 35-feature baseline drops from 76.0% to 48.8%, widening the performance gap as more models are added. This indicates that modeling the structural characteristics of encrypted traffic, rather than only aggregate timing and packet-size statistics, produces fingerprints that remain discriminative as the candidate set grows.

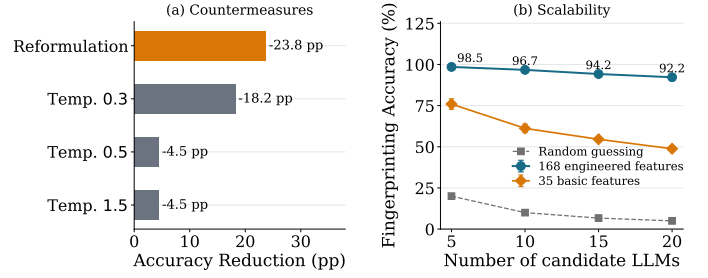


Fig. 9: (a) Accuracy reduction from prompt reformulation and decoding-temperature randomization relative to the 93.2% baseline across four prompt-grouped folds. (b) Fingerprinting accuracy for 5–20 LLMs using 168 engineered features vs. a 35-feature baseline.

VI. DISCUSSION

A. Potential Countermeasures

Countermeasures against encrypted LLM-traffic fingerprinting can be deployed at different stages of the serving pipeline. Some are specific to LLM applications and serving systems, whereas others can be adapted from the traffic-analysis and website-fingerprinting literature.

Application- and model-serving defenses. Applications can reformulate prompts through rewriting or paraphrasing while preserving the intended task. Using LLMmap’s prompt-crafting strategy reduces model-fingerprinting accuracy from 93.2% to 69.5%, showing that semantically equivalent prompts can substantially alter encrypted traffic. Providers can likewise vary decoding parameters such as temperature, top-p, top-k, or repetition penalties. Temperature variation also reduces accuracy, though non-monotonically, indicating that its effectiveness depends on the prompt and generated response. Overall, application- and model-level adaptations reduce but do not eliminate traffic fingerprintability.

Network-layer defenses. Prior traffic-analysis and website-fingerprinting defenses use padding, token grouping, response batching, dummy packets, and traffic shaping to obscure packet-size and timing patterns. These mechanisms complement application- and model-level defenses but add bandwidth, latency, or usability overhead. Because our attacks exploit correlated signals across packet sizes, timing, bursts, rates, and response progression, effective defenses must shape the overall traffic profile rather than mask a single feature.

B. Future Work

Future work should investigate defenses across all three layers. At the application layer, adversarial optimization could identify semantics-preserving prompt reformulations that minimize fingerprintability while preserving task correctness. Our prompt-reformulation results (Fig. 9(a)) suggest that this direction is promising but underexplored. At the model-serving layer, randomized decoding, token-chunk batching, and lightweight padding merit investigation, motivated by our temperature experiments. At the network layer, adaptive traffic shaping, dummy packets, and randomized delays could jointly obscure correlated traffic signals while balancing privacy gains against latency and bandwidth overhead.

VII. CONCLUSION

We present the first systematic study and benchmark of fingerprinting across direct user–LLM and collaborative multi-agent interactions. Using encrypted traces, we show that packet sizes, directions, timing, and burst structure reveal the deployed LLM, prompt category, and collaborative task despite payload encryption. These findings establish encrypted AI traffic as a measurable metadata side channel and expose privacy risks beyond message contents. The released benchmark supports reproducible evaluation of future attacks and defenses and advances the development of AI serving systems with stronger metadata privacy.

REFERENCES

- [1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” in *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [2] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, D. Z. Wang, Z. Wang, C. Qian, R. Tang, H. Ji *et al.*, “Multiagentbench: Evaluating the collaboration and competition of llm agents,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 8580–8622.
- [3] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1928–1943.
- [4] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, “Tik-tok: The utility of packet timing in website fingerprinting attacks,” *Proceedings on Privacy Enhancing Technologies*, 2020.
- [5] S. Alhazbi, A. M. Hussain, G. Oligeri, and P. Papadimitratos, “LLMs have rhythm: Fingerprinting large language models using inter-token times and network traffic analysis,” *arXiv preprint arXiv:2502.20589*, 2025.
- [6] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, “Touching from a distance: Website fingerprinting attacks and defenses,” in *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. ACM, 2012, pp. 605–616.
- [7] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, “Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail,” in *IEEE symposium on security and privacy*. IEEE, 2012, pp. 332–346.
- [8] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, “Effective attacks and provable defenses for website fingerprinting,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 143–157.
- [9] J. Hayes and G. Danezis, “k-fingerprinting: A robust scalable website fingerprinting technique,” in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1187–1203.
- [10] H. Jeong, M. Teymorianfard, A. Kumar, A. Houmansadr, and E. Bagdasarian, “Network-level prompt and trait leakage in local research agents,” *USENIX SECURITY*, 2025.
- [11] R. Weiss, D. Ayzenshteyn, and Y. Mirsky, “What was your prompt? a remote keylogging attack on AI assistants,” in *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 3367–3384.
- [12] N. Carlini and M. Nasr, “Remote timing attacks on efficient language model inference,” *arXiv preprint arXiv:2410.17175*, 2024.
- [13] D. Pasquini, E. M. Kornaropoulos, and G. Ateniese, “{LLMmap}: Fingerprinting for large language models,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 299–318.
- [14] G. McDonald and J. B. Or, “Whisper leak: a side-channel attack on large language models,” *arXiv preprint arXiv:2511.03675*, 2025.
- [15] Y. Zhang, X. Deng, Z. Gu, Y. Chen, K. Xu, Q. Li, and J. Wu, “Exposing llm user privacy via traffic fingerprint analysis: A study of privacy risks in llm agent interactions,” *arXiv preprint arXiv:2510.07176*, 2025.
- [16] R. Jansen, R. Wails, and A. Johnson, “A measurement of genuine tor traces for realistic website fingerprinting,” *arXiv preprint arXiv:2404.07892*, 2024.
- [17] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, “Holistic evaluation of language models,” *arXiv preprint arXiv:2211.09110*, 2022.
- [18] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
- [19] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “Routerbench: A benchmark for multi-llm routing system,” *arXiv preprint arXiv:2403.12031*, 2024.
- [20] H. Li, Y. Zhang, Z. Guo, C. Wang, S. Tang, Q. Zhang, Y. Chen, B. Qi, P. Ye, L. Bai *et al.*, “Llmrouterbench: A massive benchmark and unified framework for llm routing,” *arXiv preprint arXiv:2601.07206*, 2026.
- [21] A. Hintz, “Fingerprinting websites using traffic analysis,” in *International workshop on privacy enhancing technologies*. Springer, 2002, pp. 171–178.
- [22] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, “Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1131–1148. [Online]. Available: <https://doi.org/10.1145/3319535.3354217>
- [23] G. Cherubin, R. Jansen, and C. Troncoso, “Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world,” in *31st USENIX Security Symposium (USENIX Security)*. USENIX Association, 2022, pp. 753–770.
- [24] A. Bahramali, A. Bozorgi, and A. Houmansadr, “Realistic website fingerprinting by augmenting network traces,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [25] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, “Sok: A critical evaluation of efficient website fingerprinting defenses,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 969–986.
- [26] Z. Zhang, G. Wu, S. Deng, S. Wang, and Y. Zhang, “Netecho: From real-world streaming side-channels to full llm conversation recovery,” *arXiv preprint arXiv:2510.25472*, 2025.
- [27] S. Alhazbi, A. Hussain, G. Oligeri, and P. Papadimitratos, “Llms have rhythm: Fingerprinting large language models using inter-token times and network traffic analysis,” *IEEE Open Journal of the Communications Society*, 2025.
- [28] C. Song, H. Wang, and R. Martin, “Personafingerprint: Measuring persona inference on modern websites with llm-driven browsing,” *arXiv preprint arXiv:2605.15962*, 2026.
- [29] Y. Kim, C. Park, T. Kim, E. Park, S. Schmidgall, S. Rahman, C. Park, C. Breazeal, X. Liu, H. Palangi *et al.*, “Teambench: Evaluating agent coordination under enforced role separation,” *arXiv preprint arXiv:2605.07073*, 2026.
- [30] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [31] M. Du, B. Xu, C. Zhu, X. Wang, and Z. Mao, “Deepresearch bench: A comprehensive benchmark for deep research agents,” *arXiv preprint arXiv:2506.11763*, 2025.
- [32] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, “What disease does this patient have? a large-scale open domain question answering dataset from medical exams,” *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021.
- [33] N. Guha, J. Nyarko, D. Ho, C. Ré, A. Chilton, A. Chohlas-Wood, A. Peters, B. Waldon, D. Rockmore, D. Zambrano *et al.*, “Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models,” *Advances in neural information processing systems*, vol. 36, pp. 44 123–44 279, 2023.
- [34] Members of the University of Tokyo Debate Society, “English debate motions (UTDS),” Hugging Face dataset, 2022. [Online]. Available: <https://huggingface.co/datasets/kokhayas/english-debate-motions-utds>
- [35] R. Weiss, D. Ayzenshteyn, and Y. Mirsky, “What was your prompt? a remote keylogging attack on {AI} assistants,” in *USENIX Security Symposium*, 2024.
- [36] S. Li, H. Guo, and N. Hopper, “Measuring information leakage in website fingerprinting attacks and defenses,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1977–1992.

- [37] R. M. Fano and D. Hawkins, "Transmission of information: A statistical theory of communications," *American Journal of Physics*, vol. 29, no. 11, pp. 793–794, 1961.